

Fiche de révision — TI307

Introduction au système Linux

En vue du Devoir Écrit S42 (QCM, 1h, 60 %)

Périmètre : TP1 à TP4, exercices 1 à 7 (tronc commun). Exercices **étoile** (★) **exclus**.

Mode d'emploi de la fiche

- Chaque TP est résumé en **4 blocs** : objectifs Bloom → commandes/concepts clés → pièges fréquents → auto-test.
- Les **pièges** marqués ► sont les items récurrents aux QCM d'analyse (exercices de niveau « élevé » dans les énoncés).
- Référentiel : alignement Biggs (1996), objectifs Bloom (Anderson & Krathwohl, 2001).
- **Note de cadrage** : seuls les exercices 1–7 du tronc commun sont couverts ici, conformément à votre demande. Les exercices ★ (8–10) de chaque TP sont **exclus** de cette fiche.

1. TP1 — Premières commandes, navigation, gestion de fichiers

Objectifs

[Comprendre] le modèle shell / commande / arguments. **[Appliquer]** naviguer dans l'arborescence, lire et manipuler des fichiers, consulter l'aide intégrée. **[Analyser]** distinguer commandes internes (*builtins*) et commandes externes.

Commandes et concepts clés

Catégorie	Commandes essentielles
Navigation	<code>pwd</code> (répertoire courant) · <code>cd [chemin]</code> · <code>ls -l -a -R</code>
Lecture	<code>cat</code> · <code>wc -l -w -c</code> · <code>head -n N</code> · <code>tail -n N</code>
Création / copie / déplacement	<code>mkdir [-p]</code> · <code>touch</code> · <code>cp [-r]</code> · <code>mv</code> · <code>rm [-i -r]</code>
Identification	<code>file <chemin></code>
Aide	<code>man <cmd></code> (externes) · <code>help <cmd></code> (internes) · <code>-help</code>

Sections du manuel `man`

1 commandes utilisateur **2** appels système (noyau) **3** bibliothèques C **8** administration système
 Dans une page `man` : /motif pour chercher, n suivant, q quitter. Lire d'abord **NAME** puis **SYNOPSIS**.

Jokers (globbing) — expansion par le shell avant l'exécution

* : séquence quelconque (y compris vide) ? : exactement 1 caractère [abc] : classe de caractères
 Exemples : `cp *.csv data/` · `rm rapport_v[0-9].pdf` · `ls exo_0?.py`

Chemins absolus vs relatifs

Absolu : commence par / (ex. `/etc/passwd`). Relatif : par rapport à `pwd` (ex. `./script.sh`, `../dossier`). `~` = répertoire personnel.

Pièges fréquents (QCM)

À retenir avant l'examen

- ► `rm *` dans le mauvais répertoire est **irréversible** : toujours vérifier avec `echo *` ou `ls *` d'abord.
- ► `cd` sans argument ramène au **home**, pas à la racine (c'est `cd /` qui va à la racine).
- ► `help` fonctionne pour les **builtins** (`cd`, `echo`, `pwd`, `type`) ; `man` pour les **externes** (`ls`, `cp`, `mv`, `rm`, `cat`). Utiliser `type <cmd>` pour savoir laquelle.
- Le shell développe le joker **avant** de lancer la commande : `rm *.txt` dans un dossier vide produit l'erreur « No such file or directory » sur le motif littéral.

Auto-test TP1

Je sais : () exécuter `man` et identifier la bonne section ; () distinguer builtin / externe ; () utiliser les jokers sans détruire mon dossier ; () composer un chemin relatif correct depuis `pwd`.

2. TP2 — Système de fichiers et permissions

Objectifs

[Comprendre] le FHS (une seule racine `/`). **[Appliquer]** identifier le type d'un fichier, créer des liens symboliques, lire/modifier les permissions. **[Analyser]** diagnostiquer « Permission denied ». **[Évaluer]** choisir les permissions minimales pour un cas d'usage.

FHS — Filesystem Hierarchy Standard

Chemin	Contenu
<code>/bin</code>	commandes essentielles (ls, cat, cp...)
<code>/sbin</code>	commandes d'administration (fdisk, ifconfig...)
<code>/etc</code>	fichiers de configuration système
<code>/home</code>	répertoires personnels des utilisateurs
<code>/root</code>	répertoire personnel de l'administrateur
<code>/usr</code>	applications et bibliothèques (<code>/usr/bin</code> , <code>/usr/lib</code>)
<code>/var</code>	données variables (journaux, bases)
<code>/tmp</code>	fichiers temporaires
<code>/dev</code>	fichiers spéciaux représentant les périphériques
<code>/proc</code>	FS virtuel (infos sur les processus)
<code>/boot</code>	noyau et fichiers de démarrage

Types de fichiers (*premier caractère de `ls -l`*)

- fichier ordinaire **d** répertoire **l** lien symbolique **c** périph. caractère
b périph. bloc **p** tube nommé (FIFO) **s** socket

Confirmation précise : `file <chemin>` (lit le contenu, pas seulement le nom).

Liens symboliques

`ln -s cible nom_lien` crée un *raccourci*. Si la cible est supprimée, le lien devient **cassé** (*dangling*) : `cat lien` échoue avec « No such file or directory ».

Permissions — la grille à mémoriser

Lecture d'une ligne `ls -l`

```
-rw-r--r--  1 user group  215 Sep 15 10:24 notes.txt
```

Caractère 1 : type · caractères 2-4 : user (u) · 5-7 : group (g) · 8-10 : others (o).

r=4 (lecture) · **w**=2 (écriture) · **x**=1 (exécution ou *traversée* pour un répertoire).

Octal	Symbolique	Sens / usage typique
7	rwX	tout : scripts shell appartenant à l'utilisateur
6	rw-	lecture + écriture : fichiers de données personnels
5	r-X	lecture + exécution : binaires système partagés
4	r-	lecture seule
0	--	aucun droit
644	rw-r-r-	défaut fichier (umask 022) : partagé en lecture
755	rwXr-Xr-X	défaut répertoire ou binaire exécutable
600	rw----	fichier privé (clé SSH, mot de passe)
700	rwX---	répertoire privé

Pièges fréquents (QCM)

À retenir avant l'examen

- ▶ `chmod 777` = tout le monde peut lire, écrire, exécuter. **Jamais** sur un dossier public, jamais sur un script.
- ▶ **Traversée bloquée** : si un répertoire `projet/privé/` a `700`, un autre utilisateur ne peut pas lire `secret.txt` à l'intérieur, **même si** `secret.txt` a `644` — il faut le droit **x** sur chaque répertoire du chemin.
- ▶ Sur un répertoire : **r** = lister les noms, **w** = créer/supprimer des entrées, **x** = *entrer dedans* avec `cd`.
- ▶ **Lien symbolique cassé** : `ls -l` affiche encore le lien, mais toute opération (`cat`, `open`) échoue.

- `a=r` dans `chmod = u=r,g=r,o=r` (`a = all`).

Auto-test TP2

- () Je récite le rôle de `/etc`, `/home`, `/usr`, `/var`, `/dev`, `/proc`. () Je lis une ligne `ls -l` et je donne l'octal correspondant.
- () Je choisis les permissions minimales pour : (a) mot de passe personnel, (b) script d'équipe, (c) page web publique.
- () J'explique la traversée bloquée.

3. TP3 — Environnement de travail et compilateur C

Objectifs

[Comprendre] les 4 étapes de la compilation C. [Appliquer] compiler avec `gcc`, lire `argc/argv`, gérer `errno`, manipuler `open/read/write/close`. [Analyser] interpréter les `warnings gcc`.

Les 4 étapes de la compilation

#	Outil	Entrée → sortie	Option gcc
1	<code>cpp</code>	<code>.c</code> → texte étendu	<code>-E</code>
2	<code>gcc (compile)</code>	texte étendu → assembleur <code>.s</code>	<code>-S</code>
3	<code>as</code>	<code>.s</code> → objet <code>.o</code>	<code>-c</code>
4	<code>ld (link)</code>	<code>.o</code> + bibliothèques → exécutable	(aucune)

Options gcc à mémoriser

`-o nom` : nommer l'exécutable (sinon `a.out`) · `-Wall` : avertissements courants · `-Wextra` : avertissements supplémentaires · `-Werror` : warnings → erreurs · `-g` : symboles de debug.

Arguments en ligne de commande

```
int main(int argc, char **argv) { ... }
./monprog hello world
-> argc = 3, argv[0] = "./monprog", argv[1] = "hello", argv[2] = "world"
-> argv[argc] vaut toujours NULL (fin du tableau, convention execv)
```

Gestion des erreurs système

Outil	Rôle
<code>errno</code>	variable globale (dans <code><errno.h></code>) positionnée par l'appel qui échoue
<code>perror("ctx")</code>	affiche sur <code>stderr</code> : <code>ctx: <message lisible>\n</code>
<code>strerror(errno)</code>	renvoie la chaîne descriptive correspondant au code

Codes à connaître : `ENOENT` (fichier inexistant), `EACCES` (permission refusée), `EPERM` (opération non permise).

Appels système fichiers

Appel	Rôle
<code>open(path, flags, mode)</code>	ouvre un fichier, renvoie un descripteur entier · <code><fcntl.h></code>
<code>read(fd, buf, count)</code>	lit jusqu'à <code>count</code> octets ; renvoie le nombre lu, 0 à EOF, -1 à l'erreur
<code>write(fd, buf, count)</code>	écrit <code>count</code> octets ; renvoie le nombre écrit
<code>close(fd)</code>	ferme le descripteur

Trois descripteurs toujours ouverts : `0` = `STDIN_FILENO` (clavier) · `1` = `STDOUT_FILENO` (terminal) · `2` = `STDERR_FILENO` (terminal, erreurs).

Pièges fréquents (QCM)

À retenir avant l'examen

- ▶ `./programme` : le `./` est obligatoire parce que `.` (le répertoire courant) n'est **pas** dans `$PATH`.
- ▶ `gcc hello.c` sans `-o` produit `a.out`, pas `hello`.
- ▶ Compiler **sans** `-Wall -Wextra` laisse passer des bugs (variable non initialisée, format `%d` avec un `double`, variable non utilisée).
- ▶ Un tampon (`char buf[4096]`) est utilisé parce que lire 1 octet par 1 octet déclenche un appel système par

octet → coût prohibitif.

- `man 2 X` = appels système (noyau) ; `man 3 X` = bibliothèque C (ex. `man 3 printf`).

Auto-test TP3

() Je cite les 4 étapes de compilation et l'option `gcc` pour s'arrêter à chacune. () J'écris `main(int argc, char **argv)` et je sais ce que vaut `argv[0]`. () J'écris la séquence `open` → `read` → `close` avec vérification `fd < 0`. () Je nomme 3 codes `errno` courants.

4. TP4 — Redirections, processus, signaux

Objectifs

[**Appliquer**] redirections (`>`, `>>`, `<`, `1>`, `2>`), tubes (`|`), contrôle de tâches. [**Analyser**] distinguer processus vs tâche, avant-plan vs arrière-plan. [**Évaluer**] choisir entre arrêt propre (`SIGTERM`) et arrêt forcé (`SIGKILL`).

Les 3 canaux standards

#	Nom	Rôle par défaut
0	stdin	lit depuis le clavier
1	stdout	écrit sur le terminal (sortie normale)
2	stderr	écrit sur le terminal (messages d'erreur)

Redirections (opérateurs du shell)

Opérateur	Effet
<code>> fichier</code>	redirige stdout, écrase le fichier (créé s'il n'existe pas)
<code>>> fichier</code>	redirige stdout, ajoute en fin de fichier
<code>1> fichier</code>	synonyme explicite de <code>> fichier</code>
<code>2> fichier</code>	redirige stderr vers un fichier
<code>< fichier (= 0<)</code>	redirige stdin : la commande lit depuis le fichier
<code>2>&1</code>	redirige stderr vers où va stdout

Exemple canonique : `cat f1 f2 f3 1> result.txt 2> error.txt` sépare proprement résultats et erreurs.

Tubes (*pipes*)

`cmd1 | cmd2` : la sortie standard de `cmd1` devient l'entrée standard de `cmd2`. Enchaînable : `ls /usr/include/*.h | wc -l`.

Piège QCM — tube vs substitution

`ls *.h | wc -l` : `wc -l` compte les **lignes de la sortie de ls** (= nombre de fichiers).
`wc -l $(ls *.h)` : `wc -l` reçoit les **noms de fichiers** en argument et compte les lignes à l'intérieur de chacun — résultat totalement différent.

Processus et tâches

Commande	Rôle
<code>ps</code>	liste les processus de l'utilisateur courant (colonnes PID, TTY, TIME, CMD)
<code>ps -e / -A</code>	tous les processus du système
<code>top</code>	afficheur interactif, trié par CPU ; quitter avec <code>q</code>
<code>jobs</code>	liste les tâches du shell courant
<code>jobs -p</code>	affiche les PID
<code>fg %n</code>	ramène la tâche <code>n</code> au premier plan
<code>bg %n</code>	reprend la tâche <code>n</code> en arrière-plan
<code>cmd &</code>	lance <code>cmd</code> directement en arrière-plan

Raccourcis : `Ctrl+Z` suspend la tâche courante (envoie `SIGTSTP`) · `Ctrl+C` interrompt (`SIGINT`) · `Ctrl+D` envoie EOF sur stdin.

Signaux à mémoriser

Signal	Num.	Sémantique
SIGINT	2	interruption (clavier Ctrl+C) — le processus <i>peut</i> l'ignorer
SIGTSTP	20	suspension (clavier Ctrl+Z)
SIGCONT	18	reprise d'un processus suspendu (fg , bg)
SIGTERM	15	demande d'arrêt propre (défaut de kill)
SIGKILL	9	arrêt forcé — ne peut pas être ignoré ou capturé

Syntaxes équivalentes de **kill** : **kill -SIGTERM <PID>** = **kill -s SIGTERM <PID>** = **kill -15 <PID>** = **kill <PID>**.

Liste exhaustive : **kill -L**.

Pièges fréquents (QCM)

À retenir avant l'examen

- ► **> écrase sans demander** — toujours préférer **>>** pour journaliser.
- ► **ls > list_files.txt** : le fichier **list_files.txt** apparaît dedans parce que le shell le crée (via **>**) **avant** que **ls** ne lise le répertoire.
- ► **SIGKILL** (9) ne peut pas être capturé ni ignoré $\Rightarrow$ pas de *cleanup*, pas de sauvegarde $\Rightarrow$ dernier recours uniquement.
- ► Un processus **suspendu** (**Stopped** dans **jobs**) consomme encore de la mémoire ; il faut **kill %n** pour vraiment le terminer.
- ► Choix de signal : (a) interrompre son propre programme $\rightarrow$ **SIGINT** ; (b) arrêter proprement un service $\rightarrow$ **SIGTERM** ; (c) processus figé $\rightarrow$ **SIGKILL** ; (d) suspendre temporairement un calcul $\rightarrow$ **SIGTSTP**.
- On ne peut envoyer de signal **qu'à ses propres processus**, sauf **root**.

Auto-test TP4

() Je distingue **>** et **>>**, **1>** et **2>**. () J'explique **ls *.h | wc -l** vs **wc -l \$(ls *.h)**. () Je nomme les 5 signaux clés avec leur numéro. () Je choisis le bon signal dans un scénario donné.

5. Checklist finale avant l'examen

Les 20 items à cocher la veille

TP1 — Shell de base

- ☐ Je sais ouvrir une page de manuel et identifier la bonne section (1, 2, 3, 8).
- ☐ Je distingue builtin vs externe et je sais vérifier avec **type <cmd>**.
- ☐ Je lis une commande utilisant *****, **?**, **[...]** et je dis quels fichiers elle vise.
- ☐ Je compose un chemin relatif correct depuis un **pwd** donné.

TP2 — Fichiers et permissions

- ☐ Je récite les rôles de **/etc**, **/home**, **/usr**, **/var**, **/dev**, **/proc**.
- ☐ Je convertis entre notation symbolique et octale (**rw-r-r-** $\leftrightarrow$ **644**).
- ☐ Je choisis les permissions **minimales** pour un cas d'usage donné.
- ☐ J'explique pourquoi **chmod 700** sur un répertoire bloque la lecture des fichiers de **644** à l'intérieur (*traversée*).
- ☐ J'explique ce qui se passe quand on fait **cat** sur un lien symbolique cassé.

TP3 — GCC et appels système

- ☐ Je cite les 4 étapes de compilation et l'option **gcc** qui s'arrête à chacune.
- ☐ J'explique pourquoi **./programme** nécessite **./** (lien avec **\$PATH**).
- ☐ J'identifie 3 codes **errno** et je sais utiliser **perror**.
- ☐ J'écris la séquence **open-read-close** avec vérification d'erreur.
- ☐ Je justifie l'usage de **-Wall -Wextra -Werror** en 3 lignes.

TP4 — Redirections, processus, signaux

- ☐ Je distingue **>**, **>>**, **1>**, **2>**, **<**.
- ☐ Je sais tracer où va la sortie dans un pipeline à 3 étages.
- ☐ Je distingue **ls *.h | wc -l** de **wc -l \$(ls *.h)**.
- ☐ Je cite 5 signaux avec leur numéro et leur sémantique.
- ☐ Je choisis le bon signal pour : arrêter proprement / forcer / suspendre.
- ☐ Je sais basculer une tâche entre avant-plan et arrière-plan (**fg**, **bg**, **&**, **Ctrl+Z**).

Conseils de méthode pour le QCM (DE S42)

Lire la question avant les propositions

1. **Repérez le verbe Bloom** : « *quelle commande affiche* » (Appliquer) vs « *que se passe-t-il si* » (Analyser) vs « *quel est le meilleur choix* » (Évaluer). L'effort cognitif à fournir n'est pas le même.
2. **Éliminez les distracteurs grossiers** : une option avec une syntaxe inexistante (`kill -terminate`, `chmod +X`) est fautive par construction.
3. **Attention aux pièges d'ordre de redirection** : `2>&1 > fichier` n'est pas `> fichier 2>&1`.
4. **Pour les permissions** : convertissez systématiquement en octal avant de choisir.
5. **Pour les signaux** : si l'énoncé évoque un *processus figé*, le mot-clé « ne peut pas être ignoré » pointe SIGKILL.
6. **Si vous bloquez**, passez et revenez. Un QCM se joue sur le temps.

Références méthodologiques : Anderson & Krathwohl (2001), *A Taxonomy for Learning, Teaching, and Assessing*, ISBN 978-0801319037 .
Biggs, J. (1996), *Enhancing teaching through constructive alignment*, *Higher Education* 32(3), DOI:10.1007/BF00138871 .
FHS v3.0, Linux Foundation (2015) · POSIX.1-2017 (IEEE/Open Group) · GCC User Manual (FSF).
Fiche produite par Dr. Y. GUIFO FODJO — EFREI Paris — Octobre 2026.